

SynVulCommit: Validated LLM-Based Synthetic Commit Generation for Python Vulnerability Detection

Wassim Bejaoui

University of Passau

Faculty of Computer Science and Mathematics
Passau, Germany

wassim.bejaoui@uni-passau.de

Sofian Yadir

University of Passau

Faculty of Computer Science and Mathematics
Passau, Germany

sofian.yadir@uni-passau.de

Abstract

Machine-learning vulnerability detection depends on labeled code corpora, but real vulnerability-fixing commits are costly to mine and difficult to balance across weakness categories. This paper presents SynVulCommit, a provider-agnostic pipeline for generating Python vulnerability-fix commit pairs across seven VUDENC-compatible Common Weakness Enumeration (CWE) categories. The pipeline combines deterministic quota planning, CWE-specific structural checks, Bandit and Semgrep analysis, a second-stage large language model (LLM) reviewer, and exact, abstract-syntax-tree, and token-shingle duplicate rejection. The final verified window-balanced corpus contains 1,205 accepted pairs. From this corpus we freeze a balanced 1,043-pair subset with 149 synthetic commits per CWE and compare it with real VUDENC data under three training conditions: real-only, synthetic-only, and mixed real-plus-synthetic. We evaluate eight model variants from the previous labs: LSTM, multilayer perceptron (MLP), Random Forest, and convolutional neural network (CNN) with Word2Vec features, plus LSTM and CNN variants with CodeBERT and GraphCodeBERT embeddings. All models are evaluated on the same held-out real VUDENC test split. Synthetic-only training remains weaker than real-only training, showing that validated synthetic commits do not automatically match the real-data distribution. However, mixed training improves the best MLP result to a mean macro-F1 of 0.7161, the strongest overall score. These results support SynVulCommit as a controlled data-generation and augmentation framework, while showing that validation, diversity, and downstream representation must be designed together.

1 Introduction

Software vulnerabilities remain a high-impact source of security failures, and source-code learning methods are increasingly used to prioritize or classify vulnerable code. Their effectiveness depends on the quality, coverage, and labeling discipline of training data. Real repositories provide valuable examples, but vulnerability-fixing commits are sparse, noisy, and unevenly distributed across weakness types. VUDENC addresses this problem for Python by deriving vulnerable and fixed code from natural vulnerability-fixing commits [1]. However, a mined corpus cannot guarantee uniform coverage of a chosen CWE, framework, program structure, or data-flow pattern.

This work investigates whether an LLM can generate useful, commit-like Python vulnerability data while maintaining a strong validation boundary. SynVulCommit produces paired vulnerable and fixed files rather than isolated snippets. The objective is not to claim that LLM-generated data replaces real commits. Instead,

the system creates a controlled corpus that supports an explicit experiment: compare models trained on real data, synthetic data, and their combination using a shared real-data test set.

The contributions are:

- (1) A generation pipeline for seven Python CWE categories with deterministic quota-based coverage over application type, flow pattern, difficulty, and program structure.
- (2) A layered acceptance policy combining CWE-specific structural checks, Bandit, Semgrep, a blinded LLM reviewer, and near-duplicate rejection.
- (3) A clean VUDENC-style training export and a separate sanitized audit sidecar.
- (4) A controlled A/B/C evaluation using eight model variants from the earlier lab work, including Word2Vec, CodeBERT, and GraphCodeBERT representations.

2 Background and Related Work

Deep learning for vulnerability detection has progressed from code-gadget and representation-learning approaches, such as VulDeeP-ecker [2] and early deep representation learning [3], to systematic vulnerability representations [4] and graph-based program semantics [5]. Dataset construction remains central: Big-Vul demonstrates the value of commit-level vulnerability data with code changes and CVE summaries [6], while VUDENC specifically provides a Python corpus based on natural vulnerable/fixed commits [1]. Reviews of the area caution that reported performance can depend strongly on dataset construction and evaluation setup [7].

Pre-trained code models offer richer representations. CodeBERT jointly pre-trains on programming and natural-language data [8]; GraphCodeBERT adds data-flow structure [9]; and VulBERTa targets vulnerability detection through simplified source-code pre-training [10]. Lab 3 evaluated Word2Vec, CodeBERT, and GraphCodeBERT model variants and found that transformer embeddings were not consistently more stable than Word2Vec baselines. For this project we therefore include both families rather than treating transformer results as out-of-scope.

LLMs can both assist development and introduce risks. Pearce et al. found insecure code patterns in generated Copilot suggestions [11]. Conversely, LLM-based augmentation has been studied for long-tail vulnerability detection [12]. These findings motivate a conservative design: LLMs propose candidates, but deterministic analysis and independent review decide whether candidates enter the corpus.

3 SynVulCommit Methodology

3.1 Generation Scope and Quota Planning

The pipeline targets SQL injection (CWE-89), command injection (CWE-78), path traversal (CWE-22), open redirect (CWE-601), remote code execution (CWE-94), cross-site scripting (CWE-79), and cross-site request forgery (CWE-352). For each requested sample, a deterministic planner selects a compatible tuple of application type, data-flow pattern, difficulty, and structure. The planner favors the least-represented full tuple, then the least-represented marginal values, with a stable SHA-256 tie-breaker. A rejected candidate retries the same planned slot; it is not replaced by a random context.

The LLM returns a strict JSON object containing a commit message, relative filename, vulnerable code, fixed code, and exact changed lines. The prompt requires one requested CWE, a behavior-preserving fix, and no unrelated security-sensitive flow. Provider configuration is externalized through environment variables; keys, endpoints, raw replies, and authorization data are never stored in training exports.

3.2 Validation and Review

Figure 1 summarizes the acceptance path. Structural checks verify the intended source, sink, framework context, and fix strategy. Bandit and Semgrep then analyze vulnerable and fixed files. The requested rule must appear before the fix where configured, and no configured SynVulCommit CWE rule may remain after the fix. This cross-CWE post-fix rule prevents, for example, accepting an XSS fix that introduces SQL injection.

After deterministic validation, a second-stage reviewer receives the requested CWE/context and the vulnerable/fixed code, but not analyzer output or provider metadata. It returns a strict structured verdict. A candidate is accepted only when the review status is complete and the verdict is pass. This is a quality gate, not a replacement for static analysis.

3.3 Diversity, Export, and Verification

The diversity index rejects candidates in policy order: exact normalized vulnerable/fixed pair hashes, normalized AST-pair fingerprints, and same-CWE token-shingle similarity at Jaccard similarity at least 0.90. AST normalization removes identifiers, names, aliases, attributes, and literal-specific variation, preventing superficial re-naming from creating a new sample.

Accepted records are exported to `plain_<mode>` VUDENC-style files. These files retain only the commit message, vulnerable and fixed sources, diff, filename, changed-line counts, badparts, and goodparts. The separate metadata.jsonl sidecar contains sanitized provenance and validation summaries. A strict dataset verifier checks the accepted records, rejected categories, duplicate policy, reviewer state, and exact linkage between JSONL records and exported commits.

3.4 Window-Balanced Generation Profile

The first compact generation profile produced valid commit pairs, but many files were shorter than one 200-token model window. This caused most synthetic windows to overlap the vulnerable region. To align generation with the downstream VUDENC-style window

task, the final corpus uses a `window_balanced` profile. It requires realistic 420–900 token Python modules, clean context before and after the vulnerable region, localized vulnerable fragments, at least one vulnerable and one clean 200-token window, and fixed code that preserves most of the original file length. These checks are applied only to the window-balanced profile so the original compact generator remains available for diagnostics.

4 Experimental Design

4.1 Datasets and Splits

The final verified generation output contains 1,205 accepted window-balanced pairs. Figure 2 shows the per-CWE counts. SQL injection is the limiting class with 149 accepted pairs, so the experiment freezes exactly 149 synthetic commits per CWE, for 1,043 synthetic commits total. The selection is deterministic and stratified by application type, flow pattern, difficulty, and structure.

Real VUDENC data is normalized into the same commit-level representation; the legacy path-disclosure mode is mapped to directory traversal. The real-data cap is $\min(149, \text{available commits})$ per CWE. The selected real commit counts are 149 SQL, 106 command injection, 140 directory traversal, 93 open redirect, 54 remote code execution, 69 XSS, and 141 XSRF. Commits, not windows, are deterministically split approximately 70/15/15 into train, validation, and test. Consequently, every window from a commit stays in one split.

Windows contain up to 200 lexical code tokens. A vulnerable-source window overlapping a badpart is labeled vulnerable; other source windows are labeled clean. For computational tractability, each file retains at most four positive and four negative windows. Figure 3 shows that the final synthetic split is no longer positive-skewed: synthetic training contains 2,908 vulnerable and 2,898 clean windows, while synthetic validation contains 612 vulnerable and 616 clean windows.

4.2 A/B/C Conditions

Condition A trains and validates on real VUDENC data. Condition B trains and validates on the synthetic subset. Condition C trains on the union of real and synthetic training windows and validates on real VUDENC. Every condition is evaluated on the identical held-out real VUDENC test split. Synthetic test windows are prepared for audit symmetry but are not used for the main reported evaluation.

4.3 Model Families

The experiment reuses the model families from Lab 3. The Word2Vec models use a 300-dimensional Python Word2Vec representation rebuilt from the existing external Python corpus when the saved legacy vector sidecar was unavailable. LSTM and CNN consume 200-token sequences with fixed Word2Vec embeddings. MLP and Random Forest use mean-pooled Word2Vec vectors. The Word2Vec models are:

- LSTM: 100 units and 0.2 dropout;
- MLP: 256 and 128 ReLU hidden units with Adam;
- Random Forest: 300 trees and balanced class weights;
- CNN: 128 one-dimensional filters, kernel size 5, global max pooling, dense 128, and 0.2 dropout.



Rejected candidates retry the same planned context; accepted samples are verified and exported without raw provider or validation payloads.

Figure 1: SynVulCommit generation, validation, export, and evaluation pipeline. Rejected candidates retry the same planned context; only verified records enter the training export.

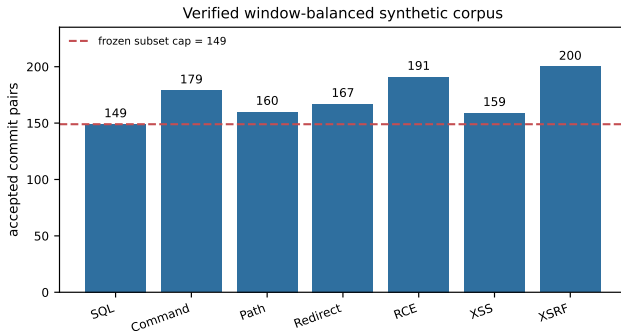


Figure 2: Verified accepted pairs in the final window-balanced corpus. The dashed line marks the 149-pair-per-CWE frozen subset used for evaluation.

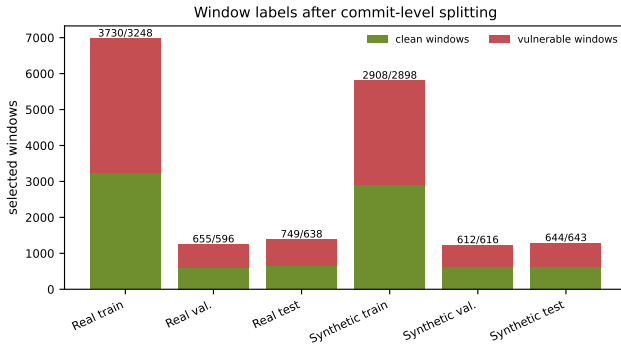


Figure 3: Selected source-window label distribution after commit-level splitting. The window-balanced generation profile produces clean and vulnerable synthetic windows in nearly equal proportions.

The transformer variants mirror the Lab 3 setup by using CodeBERT and GraphCodeBERT as frozen embedding extractors. For each 200-token window, the transformer hidden states are cached and used by either the same LSTM head or the same CNN head. The evaluated variants combine LSTM and CNN heads with CodeBERT and GraphCodeBERT embeddings. Neural models use Adam, batch

size 128, up to 100 epochs, and early stopping based on validation F1. Python, NumPy, scikit-learn, and TensorFlow seeds are fixed.

5 Results

Figure 4 reports the mean macro-F1 across the seven CWEs for all eight model variants. Macro-F1 is emphasized because a high vulnerable-class recall can hide a model that predicts most windows as vulnerable.

The strongest overall result is MLP with mixed real-plus-synthetic training, with a mean macro-F1 of 0.7161. Random Forest trained on real data alone is second at 0.6984. Mixed training improves MLP and LSTM over their real-only Word2Vec baselines, but it does not improve Random Forest or CNN. Synthetic-only training remains the weakest condition for every model family except LSTM+CodeBERT, where all three conditions are weak and close together.

The transformer variants do not outperform the best Word2Vec models. The best transformer result is CNN+GraphCodeBERT with mixed training at 0.6232 macro-F1. This is competitive with the Word2Vec LSTM and CNN results, but below MLP and Random Forest. In this setup, frozen transformer embeddings appear to help less than a simpler pooled Word2Vec MLP trained with mixed data.

Table 2 and Figure 5 list the strongest configuration for each CWE. Mixed training gives the best result for command injection, remote code execution, and XSS. Real-only training remains best for directory traversal, open redirect, SQL injection, and XSRF.

6 Discussion and Limitations

The result answers the research question conservatively. SynVulCommit can produce a validated, diverse, window-balanced synthetic corpus, and that corpus can improve some real-data models when used as augmentation. However, synthetic-only training still transfers poorly to held-out real VUDENC windows. The main remaining limitation is therefore not label imbalance; the window-balanced generation profile addressed that. The remaining limitation is distribution shift.

Real VUDENC commits come from actual projects, with inconsistent developer style, project-specific helper functions, framework conventions, partial files, and surrounding maintenance logic. Synthetic examples are cleaner and more regular, even when quota planning and duplicate rejection increase diversity. In addition, the acceptance policy deliberately favors samples that deterministic

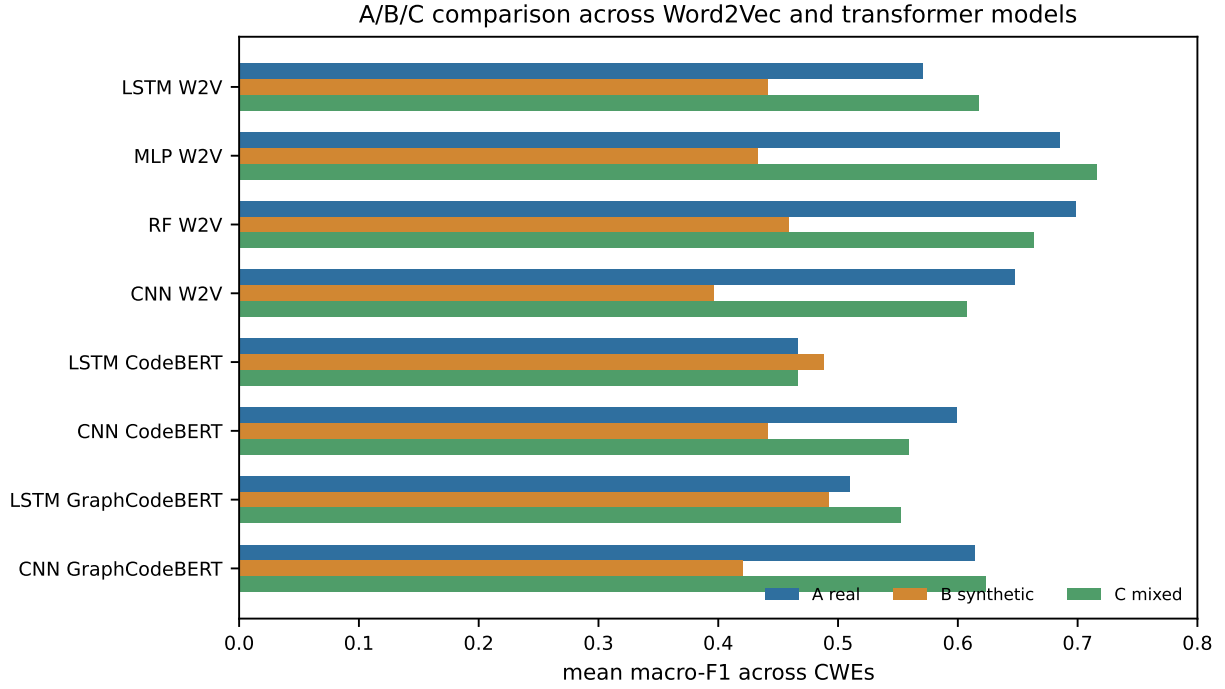


Figure 4: Mean macro-F1 across the seven CWEs for all Word2Vec and transformer model variants. All rows use the same held-out real VUDENC test split.

Table 1: Mean macro-F1 across seven CWEs by model and data condition.

Model	A: Real	B: Synthetic	C: Mixed
LSTM Word2Vec	0.5709	0.4408	0.6175
MLP Word2Vec	0.6848	0.4333	0.7161
Random Forest Word2Vec	0.6984	0.4585	0.6637
CNN Word2Vec	0.6477	0.3961	0.6073
LSTM CodeBERT	0.4663	0.4881	0.4666
CNN CodeBERT	0.5992	0.4415	0.5591
LSTM GraphCodeBERT	0.5098	0.4919	0.5524
CNN GraphCodeBERT	0.6140	0.4201	0.6232

Table 2: Best configuration per CWE on held-out real VUDENC windows.

CWE mode	Best model	Condition	Accuracy	Precision	Recall	Macro-F1
Command injection	MLP Word2Vec	C	0.8046	0.7941	0.8617	0.8014
Directory traversal	CNN Word2Vec	A	0.7366	0.7194	0.8333	0.7300
Open redirect	MLP Word2Vec	A	0.7418	0.7582	0.7340	0.7417
Remote code execution	MLP Word2Vec	C	0.7122	0.7246	0.7042	0.7122
SQL injection	MLP Word2Vec	A	0.7297	0.7286	0.8226	0.7197
XSRF	Random Forest Word2Vec	A	0.7602	0.7731	0.8107	0.7547
XSS	MLP Word2Vec	C	0.6456	0.6364	0.7000	0.6441

rules and the reviewer can verify. This improves label reliability,

but it narrows the accepted distribution toward obvious and analyzable cases. A model trained only on synthetic data may therefore

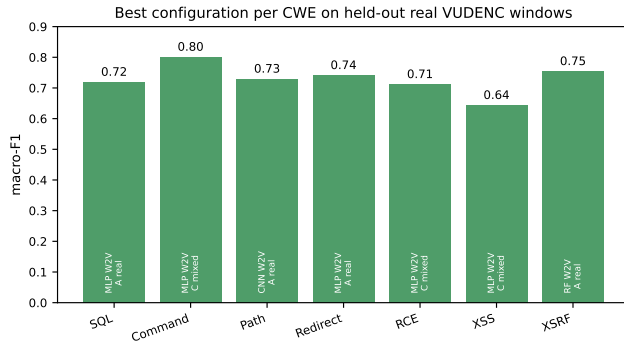


Figure 5: Best macro-F1 configuration per CWE on the held-out real VUDENC test split.

learn generated-code regularities rather than robust real-project vulnerability signals.

The transformer results reinforce this interpretation. CodeBERT and GraphCodeBERT embeddings do not automatically overcome the synthetic-to-real gap. Because the transformer encoders are used as frozen feature extractors rather than fine-tuned end-to-end, they provide richer token representations but still depend on the training distribution presented to the classifier head. The best overall result is a simpler MLP with mixed real and synthetic data, suggesting that data-condition design matters more here than representation complexity alone.

Several threats to validity remain. Static analysis and review improve confidence but do not prove exploitability or production realism. The reviewer is an LLM and can share blind spots with the generator, even when configured independently. The corpus is Python-only and covers seven CWEs, so the results should not be generalized to all languages or weakness types. The real VUDENC test split is small for remote code execution and XSS because VUDENC has fewer commits in these modes. Finally, this study uses one fixed seed for the final experiment; repeated multi-seed training would be needed to estimate variance.

7 Conclusion

SynVulCommit operationalizes controlled synthetic vulnerability-commit generation with quota-based context coverage, layered validation, reviewer gating, near-duplicate control, sanitized export, strict dataset verification, and window-balance checks aligned with the downstream VUDENC representation. The final verified corpus contains 1,205 accepted Python vulnerability-fix pairs, from which a balanced 1,043-pair subset was used for A/B/C model evaluation.

The experiments show that synthetic data alone is not a replacement for real VUDENC data in this task. However, synthetic data is useful as augmentation for some models: the best overall result is MLP with mixed real-plus-synthetic training, achieving a mean macro-F1 of 0.7161. The transformer variants from Lab 3 were also evaluated, but the best transformer model, CNN+GraphCodeBERT with mixed training, reached 0.6232 and did not outperform the strongest Word2Vec models. The practical conclusion is that validated synthetic commits can support controlled data-generation

research and improve selected model families, but generation realism and downstream representation must be treated as part of the same system.

References

- [1] Laura Wartschinski, Yannic Noller, Thomas Vogel, Timo Kehrer, and Lars Grunske. 2022. VUDENC: Vulnerability Detection with Deep Learning on a Natural Codebase for Python. *Information and Software Technology* 144 (2022), 106809. <https://doi.org/10.1016/j.infsof.2021.106809>
- [2] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *Proceedings of the Network and Distributed System Security Symposium*. <https://doi.org/10.14722/ndss.2018.23158>
- [3] Rebecca Russell, Louis Kim, Lei Hamilton, Tom Lazovich, Jacob Harer, Onur Ozdemir, Paul M. Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *2018 17th IEEE International Conference on Machine Learning and Applications*. <https://doi.org/10.1109/ICMLA.2018.00120>
- [4] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2022), 2244–2258. <https://doi.org/10.1109/TDSC.2021.3051525>
- [5] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems*.
- [6] Jiahao Fan, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories*, 508–512. <https://doi.org/10.1145/3379597.3387501>
- [7] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 48, 9 (2022), 3280–3296. <https://doi.org/10.1109/TSE.2021.3087402>
- [8] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547. <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
- [9] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, and Ming Zhou. 2021. GraphCodeBERT: Pre-Training Code Representations with Data Flow. In *International Conference on Learning Representations*.
- [10] Hazim Hanif and Sergio Maffei. 2022. VulBERTa: Simplified Source Code Pre-Training for Vulnerability Detection. In *2022 International Joint Conference on Neural Networks*, 1–8. <https://doi.org/10.1109/IJCNN55064.2022.9892280>
- [11] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *2022 IEEE Symposium on Security and Privacy*, 754–768. <https://doi.org/10.1109/SP46214.2022.9833571>
- [12] Xiao Deng, Fuyao Duan, Rui Xie, Wei Ye, and Shikun Zhang. 2024. Improving Long-Tail Vulnerability Detection Through Data Augmentation Based on Large Language Models. In *2024 IEEE International Conference on Software Maintenance and Evolution*, 262–274. <https://doi.org/10.1109/ICSME58944.2024.00033>